DATA SCIENCE

FP Growth: Frequent Pattern Generation in Data Mining with Python Implementation

In this article, an advanced method called the FP Growth algorithm will be revealed. We will walk through the whole process of the FP...

Chonyy
Oct 30, 2020 9 min read

Powerful algorithm to mine association rules in large itemsets!

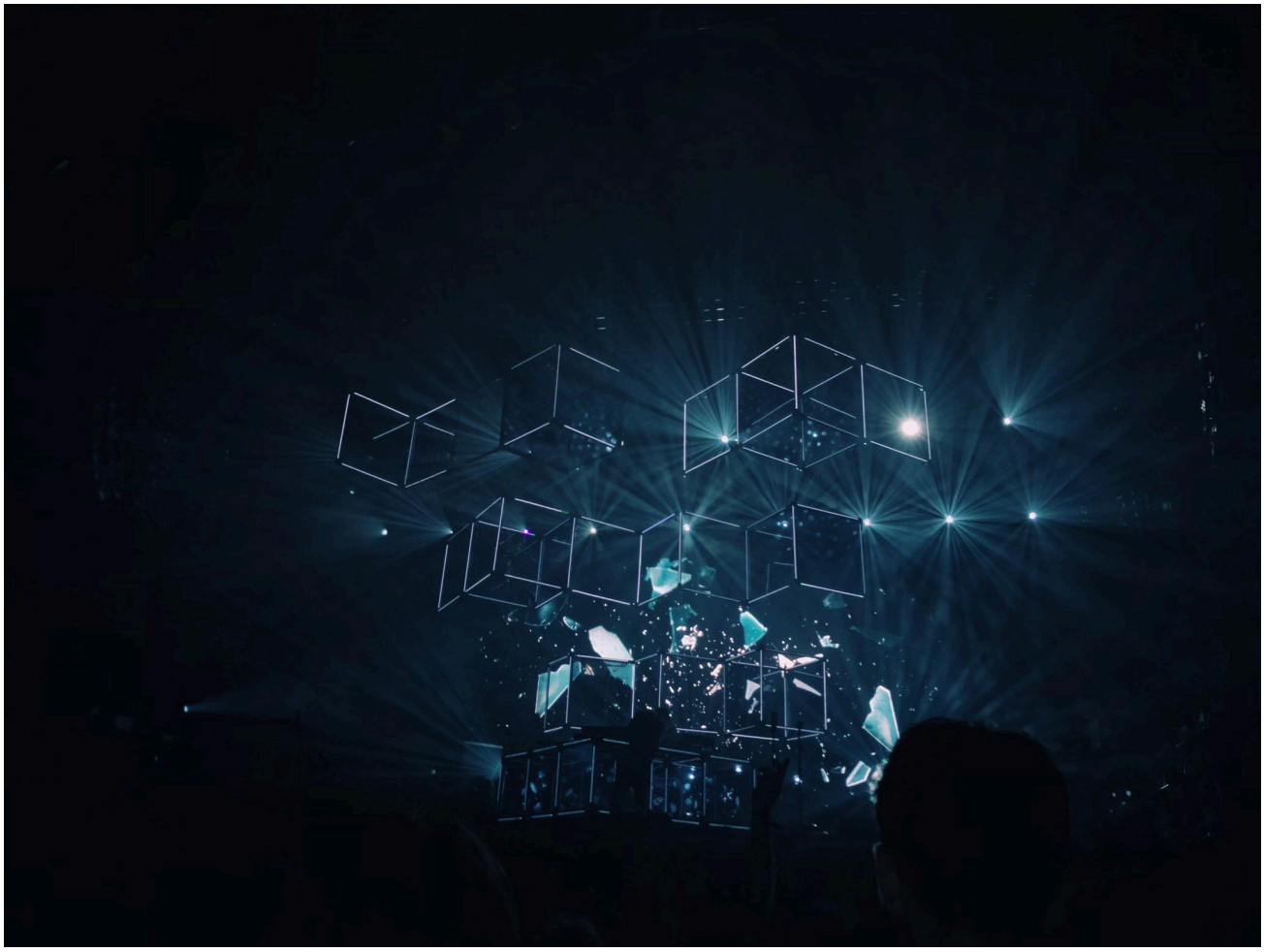


Photo by fabio on Unsplash

Introduction

We have introduced the Apriori Algorithm and pointed out its major disadvantages in the previous post. In this article, an advanced method called the FP Growth algorithm will be revealed. We will walk through the whole process of the FP Growth algorithm and explain why it's better than Apriori.

> **Apriori: Association Rule Mining In-depth Explanation and Python Implementation**

Why it's good?

Let's recall from the previous post, the two major shortcomings of the Apriori algorithm are

- The size of candidate itemsets could be extremely large
- High costs on counting support since we have to scan the itemset database over and over again

To overcome these challenges, the biggest breakthrough of Fp Growth is that

> No candidate generation is required!

All the problems of Apriori can be solved by leveraging the **FP tree**. To be more specific, the itemset size will not be a problem anymore since all the data will be stored in a way more compact version. Moreover, there's no need to scan the database over and over again. Instead, traversing the FP tree could do the same job more efficiently.

FP Tree

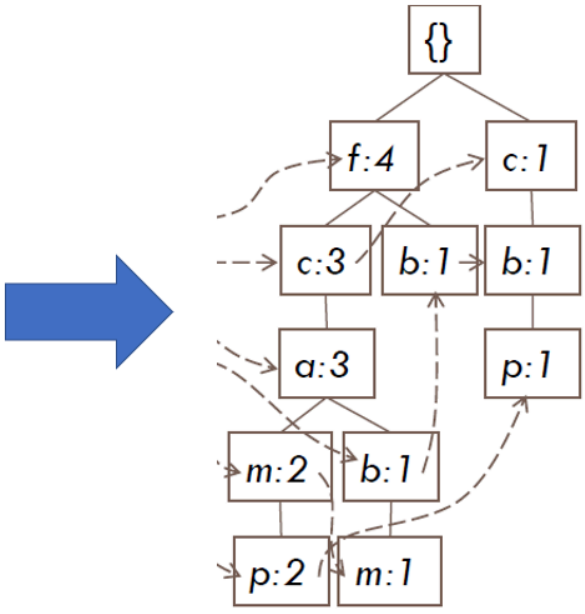
FP tree is the core concept of the whole FP Growth algorithm. Briefly speaking, the FP tree is the **compressed representation** of the itemset database. The tree structure not only reserves the itemset in DB but also keeps track of the association between itemsets

The tree is constructed by taking each itemset and mapping it to a path in the tree one at a time. The whole idea behind this construction is that

> More frequently occurring items will have better chances of sharing items

We then mine the tree recursively to get the frequent pattern. Pattern growth, the name of the algorithm, is achieved by concatenating the frequent pattern generated from the conditional FP trees.

TID	Items bought
100	{a, c, d, f, g, i, m, p}
200	{a, b, c, f, i, m, o}
300	{b, f, h, j, o}
400	{b, c, k, s, p}
500	{a, c, e, f, l, m, n, p}



Ignore the arrows on the tree. Image by Author.

FP Growth Algorithm

Feel free to check out the well-commented source code. It could really help to understand the whole algorithm.

chonyy/fpgrowth_py

The reason why FP Growth is so efficient is that it's a **divide-and-conquer** approach. And we know that an efficient algorithm must have leveraged some kind of data structure and advanced programming technique. It implemented tree, linked list, and the concept of depth-first search. The process can be split into two main stages, each stage can be further divided into two steps.

Stage 1: FP tree construction

Step 1: Cleaning and sorting

For each transaction, we first remove the items that are below the minimum support. Then, we sort the items in frequency support **descending order**.

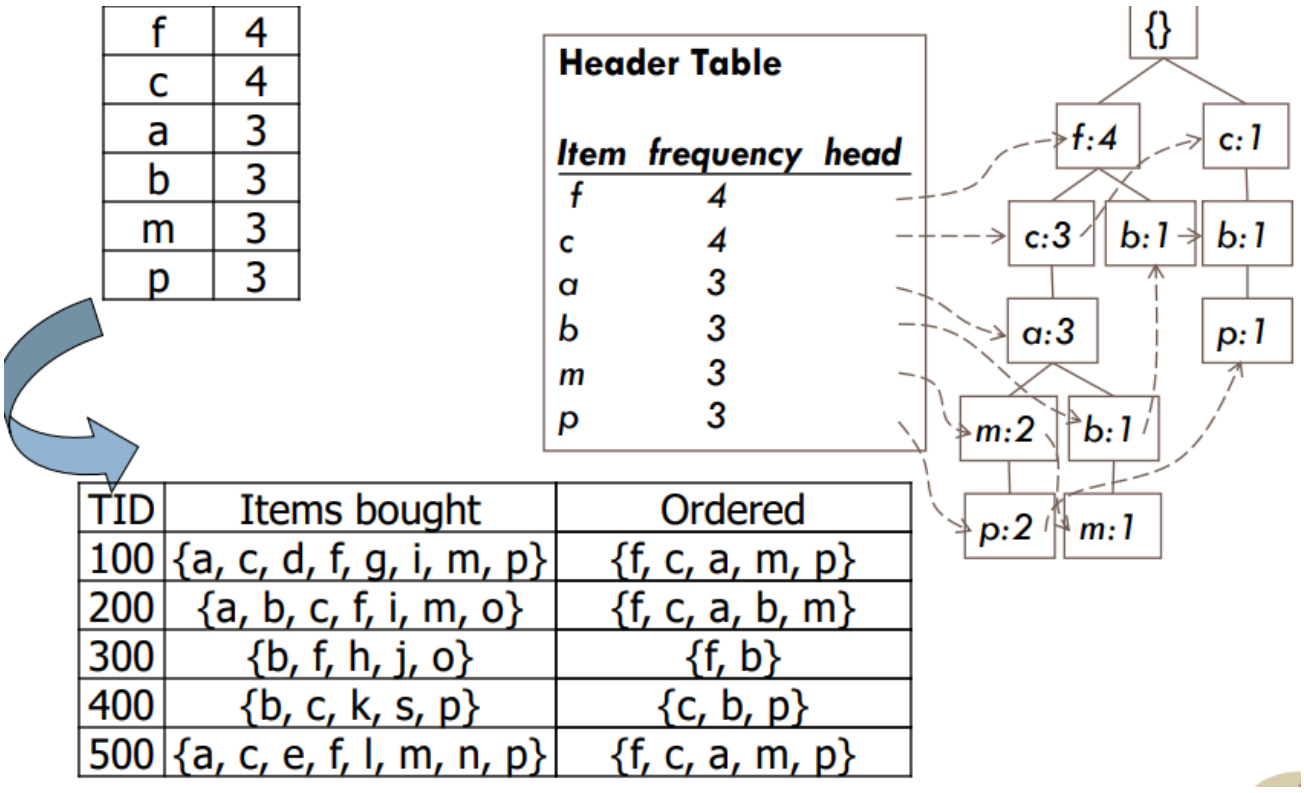
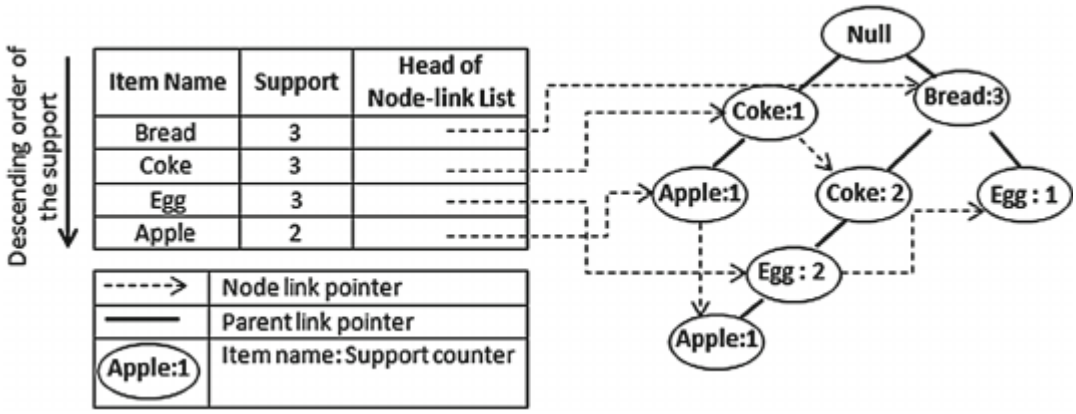


Image by Author.

Step 2: Construct FP tree, header table with cleaned itemsets

Loop through the cleaned itemsets, map it to the tree one at a time. If any item already exists in the branch, they share the same node and the count is incremented. Else, the item will be on the new branch created.

The header table is also constructed during this procedure. There's a linked list for each unique item in the transactions. With the linked list, we can find the occurrence of the item on the tree in a short period of time without traversing the tree.



Source: https://www.researchgate.net/figure/An-FP-tree-and-its-header-table-15_fig1_280940829

Stage 2: Mine the main tree and conditional FP trees

Step 1: Divide the main FP tree into conditional FP trees

Staring from each frequent 1-pattern, we create **conditional pattern bases** with **** the set of prefixes in the FP tree. Then, we use those pattern bases to construct conditional FP trees with the exact same method in Stage 1.

Step 2: Mine each conditional trees recursively

The frequent patterns are generated from the **conditional FP trees**. One conditional FP tree is created for one frequent pattern. The recursive function we used to mine the conditional

trees is close to depth-first search. It makes sure that there is no more trees can be constructed with the remaining items before moving on.

Let’s take a careful look at the process below

Image by Author.

Same level in same color. The algorithm workflow is listed below

1. Checking the first 1-frequent pattern ‘a’
2. Get the conditional FP tree for ‘a’ in pink
3. Mine the pink tree and go deeper to level 2
4. Check ‘f’ on the pink tree and found no more tree can be constructed
5. Check ‘c’ on the pink tree
6. Mine the yellow tree and go deeper to level 3

Python Implementation

FP Growth Function

```
1  def fpgrowthFromFile(fname, minSupRatio, minConf):
2      itemSetList, frequency = getFromFile(fname)
3      minSup = len(itemSetList) * minSupRatio
4      fpTree, headerTable = constructTree(itemSetList, frequency
5
6      freqItems = []
7      mineTree(headerTable, minSup, set(), freqItems)
8      rules = associationRule(freqItems, itemSetList, minConf)
9      return freqItems, rules
```

fpgrowth.py hosted with ❤ by GitHub view raw

At the first glance of this FP growth main function, you might be questioning two parts of it.

> Why using separated lists for itemset and frequency instead of making a dictionary?

The reason for it is that the key in Python dictionary has to be immutable, so we can’t take a set() as a key. However, the immutable version of set, **frozenset** is acceptable. Unfortunately, since the **order** of those items are crucial in FP growth algorithm,

and we can't retain the order after the conversion to frozenset. The only solution to this is to store it in separate lists.

> Why the FP tree is not taken as an input variable in the mineTree function?

This is the power of the header table. Since we have already stored all the occurrences in the header table, there's no need to pass the root node to the function. We can find the item on the tree rapidly with the linked list on the table.

Tree Construction

```
def constructTree(itemSetList, frequency, minSup):
    headerTable = defaultdict(int)
    # Counting frequency and create header table
    for idx, itemSet in enumerate(itemSetList):
        for item in itemSet:
            headerTable[item] += frequency[idx]

    # Deleting items below minSup
    headerTable = dict((item, sup) for item, sup in headerTable
    if(len(headerTable) == 0):
        return None, None

    # HeaderTable column [Item: [frequency, headNode]]
    for item in headerTable:
        headerTable[item] = [headerTable[item], None]

    # Init Null head node
    fpTree = Node('Null', 1, None)
    # Update FP tree for each cleaned and sorted itemSet
    for idx, itemSet in enumerate(itemSetList):
        itemSet = [item for item in itemSet if item in headerTal
        itemSet.sort(key=lambda item: headerTable[item][0], re
        # Traverse from root to leaf, update tree with given iter
        currentNode = fpTree
        for item in itemSet:
            currentNode = updateTree(item, currentNode, heade

    return fpTree, headerTable

```

```
30  def updateTree(item, treeNode, headerTable, frequency):
31      if item in treeNode.children:
32          # If the item already exists, increment the count
33          treeNode.children[item].increment(frequency)
34      else:
35          # Create a new branch
36          newItemNode = Node(item, frequency, treeNode)
37          treeNode.children[item] = newItemNode
38          # Link the new branch to header table
39          updateHeaderTable(item, newItemNode, headerTable)
40
41      return treeNode.children[item]
42
43  def updateHeaderTable(item, targetNode, headerTable):
44      if(headerTable[item][1] == None):
45          headerTable[item][1] = targetNode
46      else:
47          currentNode = headerTable[item][1]
48          # Traverse to the last node then link it to the target
49          while currentNode.next != None:
50              currentNode = currentNode.next
51          currentNode.next = targetNode
```

treeConstruction.py hosted with ❤ by GitHub view raw

With the itemset list, we first create a half-empty header table that only contains items and its frequency. We loop through each itemset, clean and sort the items in frequency **descending order**. Then, we update the tree by passing the item one by one from the cleaned itemset.

If the item already exists in the tree, we simply increment the count. Otherwise, we create a new branch with the item and attach it to the parent node. An important step to do here is to update the header table since there's a new occurrence of the item. Link the new occurrence to the linked list from header table.

Tree Mining

```
1  def mineTree(headerTable, minSup, preFix, freqItemList):
```

```
    # Sort the items with frequency and create a list
    sortedItemList = [item[0] for item in sorted(list(headerTab
    # Start with the lowest frequency
    for item in sortedItemList:
        # Pattern growth is achieved by the concatenation of su
        newFreqSet = preFix.copy()
        newFreqSet.add(item)
        freqItemList.append(newFreqSet)
        # Find all prefix path, constrcut conditional pattern bas
        conditionalPattBase, frequency = findPrefixPath(item, h
        # Construct conditonal FP Tree with conditional patterr
        conditionalTree, newHeaderTable = constructTree(condi
        if newHeaderTable != None:
            # Mining recursively on the tree
            mineTree(newHeaderTable, minSup,
                     newFreqSet, freqItemList)

def findPrefixPath(basePat, headerTable):
    # First node in linked list
    treeNode = headerTable[basePat][1]
    condPats = []
    frequency = []
    while treeNode != None:
        prefixPath = []
        # From leaf node all the way to root
        ascendFPtree(treeNode, prefixPath)
        if len(prefixPath) > 1:
            # Storing the prefix path and it's corresponding cour
            condPats.append(prefixPath[1:])
            frequency.append(treeNode.count)

        # Go to next node
        treeNode = treeNode.next
    return condPats, frequency

def ascendFPtree(node, prefixPath):
    if node.parent != None:
        prefixPath.append(node.itemName)
        ascendFPtree(node.parent, prefixPath)
```

mineTree.py hosted with ❤ by GitHub view raw

Mine Tree

Starting from the 1-frequent pattern, we find all the prefix paths and construct the conditional pattern bases with it. With the conditional pattern bases, the tree is constructed using the exact same constructTree function from above. And if the tree can be successfully constructed, we go deeper and start working on that tree. This is where the recursion happens.

Remember I mentioned that one conditional tree is constructed for each frequent pattern? You can see the pattern growth on **line 8**.

Find Prefix Path

Get the first occurrence of the item on the tree, which is the head node of the linked list in header table. Then, traverse the tree all the way up to the root to get the prefix path. After that, we go to the next node in linked list and repeat traversing.

Ascend FP Tree

Ascend the tree with self-recursion. Keep appending the items and calling itself until it reaches the root.

Comparison

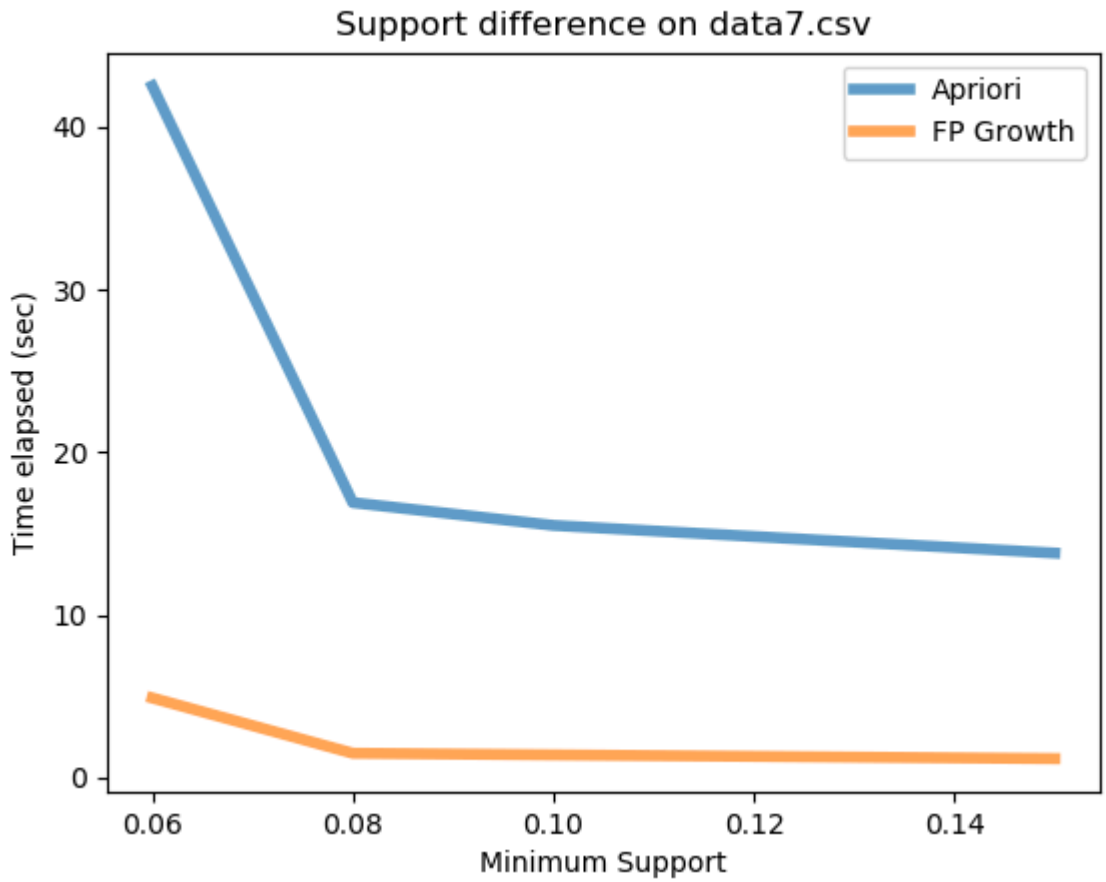
FP Growth vs Apriori

	FP Growth	Apriori
Speed	Faster, runtime increases linearly with increase in number of itemsets	Slower, runtime increases exponentially with increase in number of itemsets
Memory	Small, storing the compact version of database	Large, all the candidates from self-joining are stored in the memory
Candidates	No candidate generation	Use self-joining for candidate generation
Frequent patterns	Pattern growth achieved by mining conditional FP trees.	Patterns selected from the candidates whose support is higher than minSup.
Scans	Only require two scans	Scan the database over and over again.

Image by Author.

I have organized the main features of both algorithms and made it into the table above. From observing the table, we can tell that FP Growth is generally better than Apriori under most of the circumstances. That's why Apriori is just a fundamental method, and FP Growth is an improvement of it.

Runtime with different minimum support



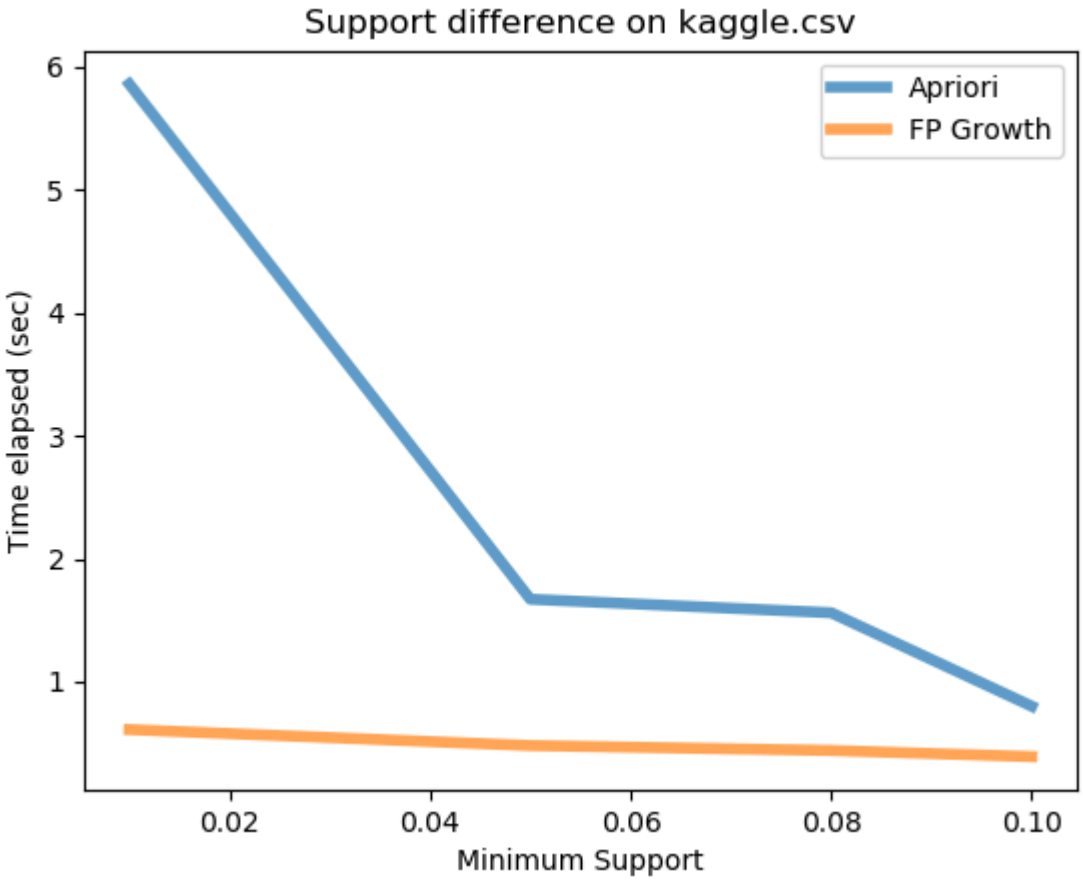
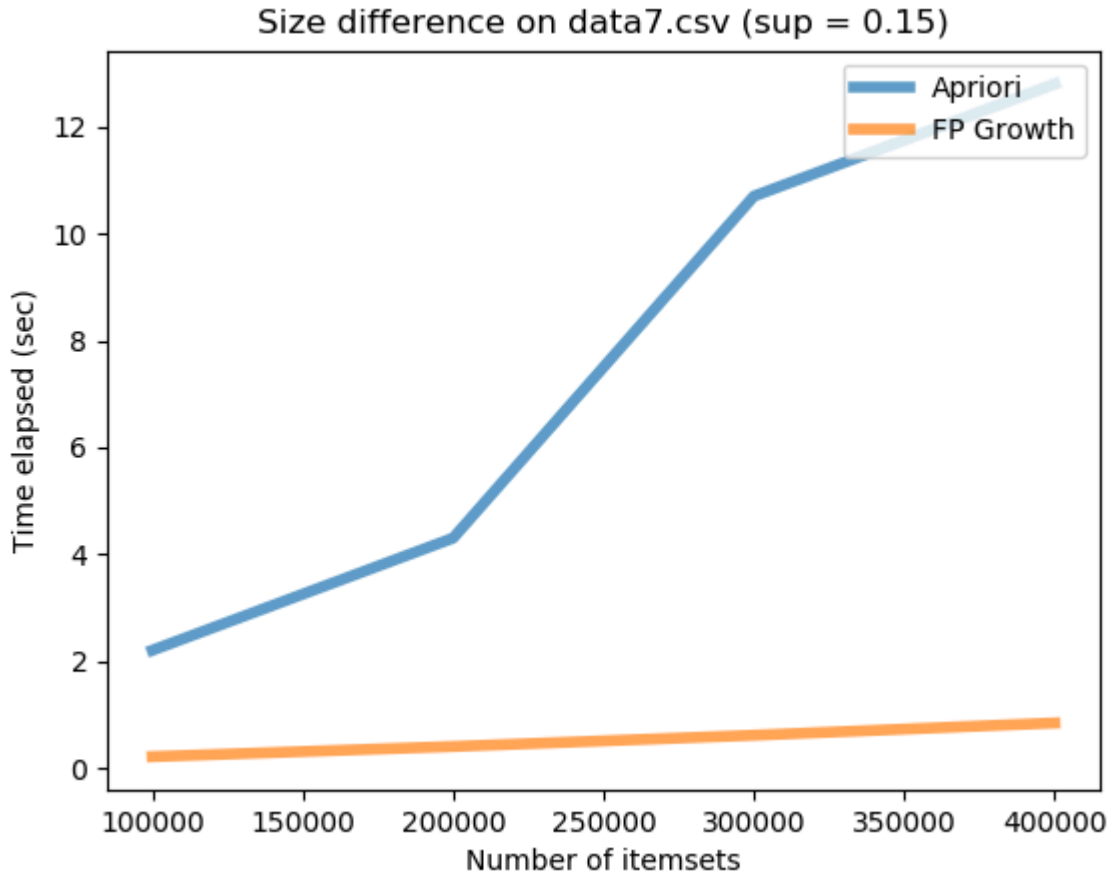


Image by Author.

We take the dataset from the **_source code repo_** and try some experiments. The first thing we do is to check how the minimum support infer the runtime.

From the plot, we can see that FP Growth is always faster than Apriori. The reason for this is already explained above. An interesting observation is that, on both datasets, the runtime of apriori started to increase rapidly after a specific minimum. On the other hand, the runtime of FP Growth is merely interfered by the value.

Runtime with different number of transaction



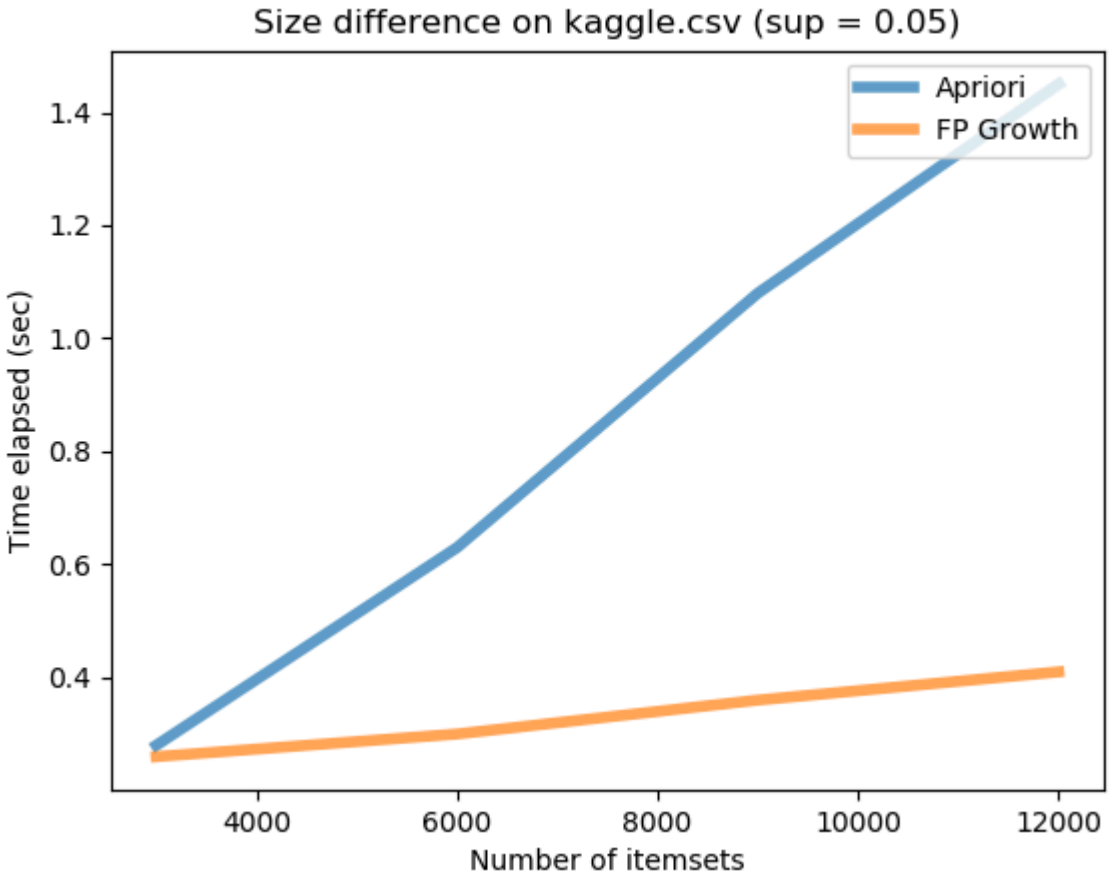


Image by Author.

The runtime of both algorithm increases as the number of itemsets goes up. However, we can clearly see that the slope of the increase of Apriori is way higher. This means that the Apriori algorithm is more sensitive to the itemsets size comparing to Fp Growth.

Ascending order vs Decreasing order

You might be wondering why we have to sort the items in frequency **descending order** before using it to construct the tree. What will happen if try it with ascending order? As you can see from the graph, sorting with descending order is always faster. And the difference between the speed is more obvious with lower support.

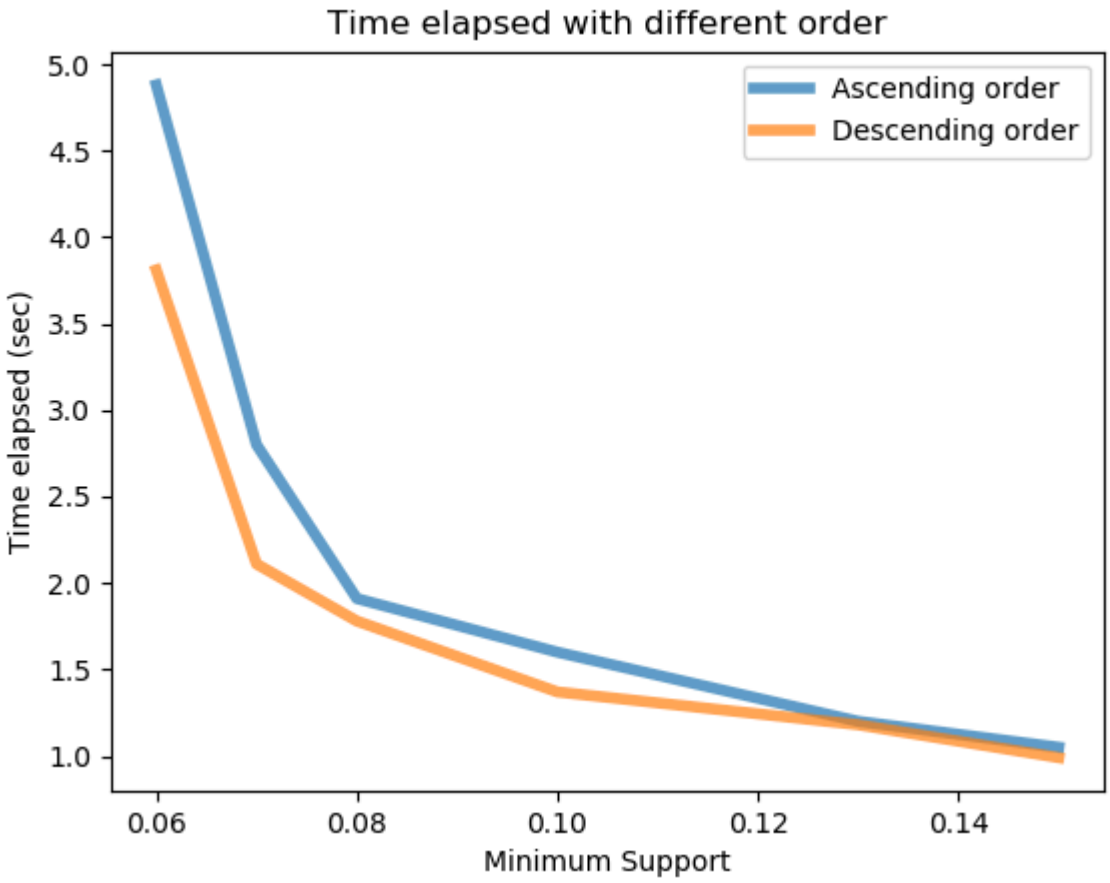


Image by Author.

But why? I will like to give some of my own assumptions. Let's recall the core concept of FP Growth

> More frequently occurring items will have better chances of sharing items

Starting from items with higher support will have a higher possibility of sharing branches. The more shared branches, the smaller is the size of the tree. And the smaller is the tree, the costs of running the whole algorithm will be less. Therefore, sorting the items in descending order will cause this method to run faster.

Conclusion

I would like to share an interesting story. While I was writing this article after work, a full-time engineer walked by and he saw that I was writing something about Apriori and Fp Growth. He said, "**Interesting, but not realistic**." He further explained that this algorithm doesn't take weighing under consideration. For example, what about a transaction with multiple same items? There are also more subtle conditions that are not included in this algorithm. And that's why companies won't implement this in their business.

Source Code

> **chonyy/fpgrowth_py**

Previous Post

> **Apriori: Association Rule Mining In-depth Explanation and Python Implementation**